PYTHON TEKSTAC SOLTIONS

1) Welcome Message

Welcome.py

```
Print(“Welcome to the Data science Course”)
```

2) Virtual Assistant

Assistant.py

```
name = input("Enter your name: ")
colour = input("Enter your colour: ")
hobby = input("Enter your hobby: ")

print(f'hello {name}! Nice to meet you.')
print(f"I see that your favourite colour is {colour}, and your favourite hobby is {hobby}. It's great to learn more about you")
```

3) Hospital Billing

Main.py

```
name = input("Enter your name: ")
ill = input("Illness: ")
adm_fee = float(input("Amission fee: "))
doc_fee = float(input("Doctor fee: "))
nur_fee = float(input("Nursing fee: "))
med_cost = float(input("Cost of medicines: "))
total = adm_fee + doc_fee + nur_fee + med_cost

print("AUMA Hospital")
print(f'Patient Name: {name}')
print(f'Illness: {ill}')
print(f'Admission fee: ${adm_fee:.2f}')
print(f'Doctor fee: ${doc_fee:.2f}')
print(f'Nursing fee: ${nur_fee:.2f}')
print(f'Cost for medicines:${med_cost:.2f}')
print(f'Total amount ${total:.2f}')
```

4) Stationary Shop

Stationary.py

```
# Get the cost of the A4 sheet from the user
sheet = float(input("Cost of A4sheet:"))

# Check if the input is negative. If it is, print an error and stop.
if sheet <= 0:
    print("Invalid input")
else:
    # If the first input is valid, proceed to the next item
    pen = float(input("Cost of pen:"))
    if pen <= 0:
        print("Invalid input")
    else:
        # If the second input is valid, proceed to the next item
        pencil = float(input("Cost of pencil:"))
        if pencil <= 0:
            print("Invalid input")
        else:
            # If the third input is valid, proceed to the next item
            eraser = float(input("Cost of eraser:"))
            if eraser <= 0:
                print("Invalid input")
            else:
                # If all inputs are valid, print the final price list
                print("Items Details")
                # The ".2f" specifier formats the float to two decimal places
                print(f"A4sheet:{sheet:.2f}")
                print(f"Pen:{pen:.2f}")
                print(f"Pencil:{pencil:.2f}")
                print(f"Eraser:{eraser:.2f}")
```

5) Theme Park Pricing

Themepark.py

```
# Get the visitor's age and convert it to an integer
age = int(input("Enter your age: "))

# First, check for invalid negative input
if age < 0:
    print("Invalid Input")
```

```
# Check if the age falls into the Child category (0-12)
elif 0 <= age <= 12:
    print("Ticket Details")
    print("Type: Child")
    # Format the price to two decimal places
    print(f"Ticket Price: Rs.{100.00:.2f}")

# Check if the age falls into the Adult category (13-64)
elif 13 <= age <= 64:
    print("Ticket Details")
    print("Type: Adult")
    print(f"Ticket Price: Rs.{200.00:.2f}")

# Any other valid age must be a Senior (65+)
else:
    print("Ticket Details")
    print("Type: Senior")
    print(f"Ticket Price: Rs.{150.00:.2f}")
```

6) Cash Exchange

Main.py

```
# Get user input for amount
amount = int(input("Enter amount: "))

# Validate amount
if amount <= 0:
    print("Invalid input")
else:
    # Get user input for denomination choice
    choice = int(input("Enter denomination choice: "))

    # Determine denomination based on choice
    if choice == 1:
        denomination = 100
    elif choice == 2:
        denomination = 200
    elif choice == 3:
        denomination = 500
    else:
```

```
        print("Invalid input")
        exit()

    # Check if amount is less than denomination
    if amount < denomination:
        print(f"Balance amount of Rs.{amount} has been returned.")
    else:
        notes = amount // denomination
        balance = amount % denomination
        print(f"{notes}*Rs.{denomination} notes have been released in exchange for
Rs.{amount}. Balance amount of Rs.{balance} has been returned.")
```

7) House Rent Calculator

Main.py

```
total_rent = 0  # To keep track of total rent from all tenants

while True:
    house_type = int(input("Enter the type of house (1 BHK / 2 BHK / 3 BHK) or 0 to quit: "))

    if house_type == 0:
        break  # Exit the loop

    # Determine base rent
    if house_type == 1:
        base_rent = 1000
    elif house_type == 2:
        base_rent = 1500
    elif house_type == 3:
        base_rent = 2000
    else:
        print("Invalid house type. Please enter 1, 2, 3 or 0 to quit.")
        continue  # Skip to next iteration

    # Get additional service preferences
    parking = int(input("Parking required? (1/0): "))
    cleaning = int(input("Special cleaning services required? (1/0): "))

    # Calculate additional charges
    if parking == 1 and cleaning == 1:
        additional = 150
    elif parking == 1:
        additional = 100
```

```
    elif cleaning == 1:
        additional = 50
    else:
        additional = 0

    # Calculate total rent for current tenant
    rent = base_rent + additional
    print(f"Rent after including additional charges: {rent}")

    # Add to total rent
    total_rent += rent

# Final output
print(f"Total rent: {total_rent}")
print("Thank you for using our application!")
```

8) Airline Reservation System

Main.py

```
available_seats = 50
booked_seats = 0
boarding_passes = []

print("Available Seats:", available_seats)
print("Business Class: Rs.15000 per seat")
print("Economy Class: Rs.7500 per seat")

while True:
    # Validate class input
    while True:
        class_choice = input("Enter class (Business/Economy) for reservation: ").strip()
        if class_choice.lower() in ['business', 'economy']:
            break
        else:
            print("Invalid class choice. Please enter Business or Economy.")

    # Validate seat input
    seat_input = input("Enter the number of seats to reserve: ").strip()
    if not seat_input.isdigit():
        print("Invalid input. Please enter a valid number.")
        continue

    seats_to_reserve = int(seat_input)
```

```
    if seats_to_reserve > available_seats:
        print(f"Invalid number of seats. Available seats: {available_seats}")
    else:
        # Calculate cost
        if class_choice.lower() == 'business':
            cost_per_seat = 15000
        else:
            cost_per_seat = 7500

        total_cost = seats_to_reserve * cost_per_seat
        print(f"Reservation successful! Total cost: Rs.{total_cost}")

        # Update seat tracking
        for i in range(booked_seats + 1, booked_seats + seats_to_reserve + 1):
            boarding_passes.append(f"Seat {i} - Boarding Pass")

        booked_seats += seats_to_reserve
        available_seats -= seats_to_reserve

    # Ask for another reservation
    another = input("Do you want to make another reservation? (yes/no): ").strip().lower()
    if another != 'yes':
        break

# Final output
print("\nBoarding Passes:")
for pass_info in boarding_passes:
    print(pass_info)

print("\nThank you for using our reservation system!")
```

9) Voting System

Main.py

```
# Initialize vote counters
alice_votes = 0
bob_votes = 0
charlie_votes = 0

# Get number of voters
num_voters = int(input("Enter the number of voters (1-100): "))
```

```
# Voting loop
for i in range(1, num_voters + 1):
    while True:
        vote = input(f"Enter the vote for voter {i} (Alice, Bob, or Charlie): ").strip()
        if vote.lower() == "alice":
            alice_votes += 1
            break
        elif vote.lower() == "bob":
            bob_votes += 1
            break
        elif vote.lower() == "charlie":
            charlie_votes += 1
            break
        else:
            print("Invalid vote. Please enter a valid vote (Alice, Bob, or Charlie).")

# Calculate percentages
alice_percent = (alice_votes / num_voters) * 100
bob_percent = (bob_votes / num_voters) * 100
charlie_percent = (charlie_votes / num_voters) * 100

# Display results
print("\nVoting Results:")
print(f"Candidate Alice received {alice_percent:.2f}% of the votes.")
print(f"Candidate Bob received {bob_percent:.2f}% of the votes.")
print(f"Candidate Charlie received {charlie_percent:.2f}% of the votes.")

# Determine winner
if (alice_votes > bob_votes) and (alice_votes > charlie_votes):
    print("\nWinner: Candidate Alice")
elif (bob_votes > alice_votes) and (bob_votes > charlie_votes):
    print("\nWinner: Candidate Bob")
elif (charlie_votes > alice_votes) and (charlie_votes > bob_votes):
    print("\nWinner: Candidate Charlie")
else:
    print("\nIt's a tie! No clear winner.")
```

10) Fitness Tracker Analyzer

Main.py

```
# Get number of days
num_days = int(input("Enter the number of days: "))
```

```
if num_days <= 0:
    print("Invalid input: No data entered.")
else:
    steps = []

    # Collect step data
    for i in range(1, num_days + 1):
        count = int(input(f"Enter the number of steps for Day {i}: "))
        steps.append(count)

    # Calculate average
    average_steps = sum(steps) / num_days

    # Find max and min
    max_steps = max(steps)
    min_steps = min(steps)
    max_day = steps.index(max_steps) + 1
    min_day = steps.index(min_steps) + 1

    # Check for increasing streak
    increasing_streak = all(steps[i] < steps[i + 1] for i in range(num_days - 1))
    decreasing_streak = all(steps[i] > steps[i + 1] for i in range(num_days - 1))

    # Display results
    print("\nAnalysis Results:")
    print(f"Average steps per day: {average_steps:.2f}")
    print(f"Day with the most steps: Day {max_day} ({max_steps} steps)")
    print(f"Day with the least steps: Day {min_day} ({min_steps} steps)")
    print(f"Consecutive increasing steps streak: {'Yes' if increasing_streak else 'No'}")
    print(f"Consecutive decreasing steps streak: {'Yes' if decreasing_streak else 'No'}")
```

11) Non-Working Doctors

Doctors.py

```
# Read size of first list
n1 = int(input())
if n1 <= 0:
    print("Invalid list size")
    exit()

# Read first list (all doctors)
all_doctors = []
for _ in range(n1):
```

```
    doc_id = int(input())
    if doc_id <= 0:
        print("Invalid ID")
        exit()
    all_doctors.append(doc_id)

# Read size of second list
n2 = int(input())
if n2 <= 0:
    print("Invalid list size")
    exit()

# Read second list (working doctors)
working_doctors = []
for _ in range(n2):
    doc_id = int(input())
    if doc_id <= 0:
        print("Invalid ID")
        exit()
    working_doctors.append(doc_id)

# Find non-working doctors
non_working = [doc for doc in all_doctors if doc not in working_doctors]

# Display result
print("Not working Doctors' IDs are:")
print(non_working)
```

12) Product Sales

Main.py

```
def main():
    # Get number of employees
    num_employees = int(input("Enter no. of employees: "))

    if num_employees <= 0:
        print("Invalid input")
        return

    # List to store employee details
    employee_data = []
```

```
    # Collect employee details
    for _ in range(num_employees):
        name = input("Enter name: ")
        sales = int(input("Enter no of products sold: "))
        city = input("Enter city: ")

        employee_data.append({'name': name, 'sales': sales, 'city': city})

    # Group data by city
    grouped_data = {}

    for entry in employee_data:
        city = entry['city']
        person = {'name': entry['name'], 'sales': entry['sales']}

        if city not in grouped_data:
            grouped_data[city] = [person]
        else:
            grouped_data[city].append(person)

    # Display grouped dictionary
    print(grouped_data)


if __name__ == '__main__':
    main()
```

13) Participant Registration and Preference Analysis

Main.py

```
def main():
    # Get number of participants
    num_participants = int(input("Enter the number of participants: "))

    if num_participants <= 0:
        print("Invalid Input")
        return

    # Dictionaries to store preferences and event participants
    participant_preferences = {}
    event_participants = {}
```

```
    # Valid event options
    valid_events = {"Workshop", "Presentation", "Hackathon", "Quiz"}

    # Collect participant data
    for _ in range(num_participants):
        name = input("Enter participant name: ")
        email = input("Enter participant email: ")
        reg_date = input("Enter registration date (YYYY-MM-DD): ")
        preferences = input("Enter participant preferences separated by spaces
(Workshop/Presentation/Hackathon/Quiz): ").split()

        # Store participant preferences
        participant_preferences[name] = preferences

        # Update event participants
        for event in preferences:
            if event not in event_participants:
                event_participants[event] = [name]
            else:
                event_participants[event].append(name)

    print("\nRegistered Successfully!")
    print("Participant Preferences:", participant_preferences)
    print("Event Participants:", event_participants)


if __name__ == '__main__':
    main()
```

14) Student Electives

Main.py

```
def main():
    # Get number of students
    num_students = int(input("Enter the no. of students: "))

    if num_students <= 0:
        print("Invalid input")
        return

    # List to store each student's electives
    student_electives = []
```

```
    # Dictionary to count each elective
    elective_count = {}

    # Collect electives for each student
    for _ in range(num_students):
        electives_input = input("Enter electives: ")
        electives = electives_input.split('/')
        student_electives.append(electives)

        # Count each elective
        for subject in electives:
            if subject in elective_count:
                elective_count[subject] += 1
            else:
                elective_count[subject] = 1

    # Display student electives list
    print("\nStudent electives list:", student_electives)

    # Display elective count
    print("\nElectives count:", elective_count)

    # Find common electives
    common_electives = set(student_electives[0])
    for electives in student_electives[1:]:
        common_electives &= set(electives)

    if common_electives:
        print("\nCommon elective:", sorted(list(common_electives)))
    else:
        print("\nNo common electives found")


if __name__ == '__main__':
    main()
```

15) Sort the Student Details Dictionary

students_detail.py

```
def display_students(students):
    for student in students:
        print(student)
```

```
def main():
    students = []

    while True:
        try:
            count = int(input("Enter the no of student details to be created: "))
        except ValueError:
            print("Invalid Input")
            return

        if count <= 0:
            print("Invalid Input")
            return

        for _ in range(count):
            name = input("Name : ").strip()
            try:
                age = int(input("Age : "))
            except ValueError:
                print("Invalid Input")
                print("Here's the list of student details :")
                display_students(students)
                print("Here's the list of student details sorted with respect to name :")
                display_students(sorted(students, key=lambda x: x['Name']))
                return

            if age < 10 or age > 80:
                print("Invalid Input")
                print("Here's the list of student details :")
                display_students(students)
                print("Here's the list of student details sorted with respect to name :")
                display_students(sorted(students, key=lambda x: x['Name']))
                return

            students.append({'Name': name, 'Age': age})

        choice = input("Do you want to add more students' details to the list of dictionaries? If yes, press 1, else press 0: ").strip()
        if choice == '1':
            continue
        elif choice == '0':
            print("Here's the list of student details :")
```

```
            display_students(students)
            print("Here's the list of student details sorted with respect to name :")
            display_students(sorted(students, key=lambda x: x['Name']))
            break
        else:
            print("Invalid Input")
            print("Here's the list of student details :")
            display_students(students)
            print("Here's the list of student details sorted with respect to name :")
            display_students(sorted(students, key=lambda x: x['Name']))
            break

if __name__ == "__main__":
    main()
```

16) Sort the Student Details Dictionary

Students_details.py

```
def main():

    student_list = []

    while True:

        try:

            num_students = int(input("Enter the no of student details to be created: "))

        except ValueError:

            print("Invalid Input")

            continue

        if num_students <= 0:

            print("Invalid Input")

            continue

        for _ in range(num_students):
```

```
        name = input("Name: ").strip()

        try:

            age = int(input("Age: ").strip())

        except ValueError:

            print("Invalid Input")

            continue

        if age < 10 or age > 50:

            print("Invalid Input")

            continue

        student_list.append({'Name': name, 'Age': age})

    choice = input("Do you want to add more student details to the list of dictionaries? If yes, press 1, else press e: ").strip()

    if choice == '1':
        continue
    elif choice.lower() == 'e':
        break
    else:
        print("Invalid Input")
print("\nHere's the list of student details:")
for student in student_list:
    print(student)
sorted_list = sorted(student_list, key=lambda x: x['Name'])
print("\nHere's the list of student details sorted with respect to name:")
for student in sorted_list:
    print(student)
if __name__ == "__main__":
    main()
```

17) Generate Customer Id

Main.py

```
def main():
    num_customers = int(input("Enter the number of customers: "))

    if num_customers <= 0:
        print("Invalid input")
        return

    for i in range(num_customers):
        details = input("Enter the customer's details: ").strip()

        try:
            name, dob, location = details.split('/')
        except ValueError:
            print("Invalid input format")
            continue

        # Reverse name and remove spaces
        reversed_name = name.replace(" ", "")[::-1]

        # Extract even index letters from reversed name
        even_letters = ''.join([reversed_name[j] for j in range(0, len(reversed_name), 2)])

        # Extract year from DOB
        year = dob.split('-')[0]

        # First and last letters of location
        loc_code = location[0] + location[-1]

        # Generate customer ID
        customer_id = even_letters + year + loc_code + str(i)

        print(f"Customer Id of {name} is {customer_id}")


if __name__ == '__main__':
    main()
```

18) Online Shopping Cart System

Main.py

```
def add_to_cart(cart, item, fixed_price, quantity):
    """Adds an item to the shopping cart or updates its quantity."""
```

```
    if item not in cart:
        # Add the new item with its price and quantity
        cart[item] = {'price': fixed_price, 'quantity': quantity}
        return "Item added to the cart"
    else:
        # Update the quantity of the existing item
        cart[item]['quantity'] += quantity
        return f"Quantity updated for {item} in the cart"

def view_cart(cart):
    """Displays the items currently in the shopping cart."""
    if not cart:
        print("No items in the cart")
    else:
        print("Items in the Cart:")
        for item, details in cart.items():
            # Display each item with its quantity and price
            print(f"{item}: {details['quantity']} x Rs.{details['price']} each")

def calculate_total(cart):
    """Calculates the total price of all items in the cart."""
    total_price = 0.0
    for item, details in cart.items():
        # Sum the price multiplied by the quantity for each item
        total_price += details['price'] * details['quantity']
    return total_price

def main():
    """Main function to run the shopping cart application."""
    # Predefined dictionary of available items and their fixed prices
    items_prices = {
        'Laptop': 48000.0,
        'Mouse': 1000.0,
        'Keyboard': 35000.0,
        'Headphones': 2000.0,
        'Printer': 15000.0
    }

    # Initialize an empty dictionary for the user's shopping cart
    shopping_cart = {}

    # Main loop to display the menu and handle user choices
    while True:
```

```
        print("\nItems: Laptop, Mouse, Keyboard, Headphones, Printer")
        print("1. Add item to cart")
        print("2. View cart")
        print("3. Calculate total")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == '1':
            item_name = input("Enter the item name: ")

            # Check if the entered item is valid
            if item_name not in items_prices:
                print("Invalid item name. Item not added to the cart")
            else:
                quantity = int(input("Enter the quantity: "))
                fixed_price = items_prices[item_name]
                # Call the function to add/update the item and print the result
                message = add_to_cart(shopping_cart, item_name, fixed_price, quantity)
                print(message)

        elif choice == '2':
            # Call the function to display the cart contents
            view_cart(shopping_cart)

        elif choice == '3':
            # Call the function to calculate the total and print it
            total = calculate_total(shopping_cart)
            print(f"Total Price: Rs. {total}")

        elif choice == '4':
            # Exit the program
            print("Thank you!")
            break

        else:
            # Handle invalid menu choices
            print("Invalid choice")

# Entry point of the script
if __name__ == "__main__":
    main()
```

19) Pattern Generator

Main.py

```
# Imports the function from the other file (Fix for Fail 1)
from pattern import generate_pattern

def main():
    """
    Main function to get user input and display the generated pattern.
    """
    input_text = input("Enter the input text: ")
    row_length = int(input("Enter the length of each row: "))

    # Call the imported function
    result_pattern = generate_pattern(input_text, row_length)

    # Print the result
    print(result_pattern)

# Entry point for the script
if __name__ == "__main__":
    main()
```

Pattern.py

```
def generate_pattern(input_pattern, length):
    """
    Generates a text/numeric pattern based on the input string and row length.
    """
    # Checks for zero or negative length (Fix for Fail 3 & 4)
    if length <= 0:
        return "Invalid input. Please provide a positive row length"

    pattern_rows = []
    pattern_len = len(input_pattern)
    start_index = 0

    # Generates the pattern in a circular manner (Fix for Fail 2)
    for _ in range(length):
        current_row = ""
        for j in range(length):
            char_index = (start_index + j) % pattern_len
            current_row += input_pattern[char_index]
```

```
        pattern_rows.append(current_row)
        start_index = (start_index + 1) % pattern_len

    final_pattern = "\n".join(pattern_rows)
    return final_pattern.strip()
```

20) Vehicle Premium

Vehicle.py

```
class Vehicle:
    def __init__(self, cost, type):
        self.__cost = cost
        self.__type = type
        self.__premium = None

    def calculate_premium(self):
        if self.__type == 1:
            self.__premium = self.__cost * 0.02
        elif self.__type == 2:
            self.__premium = self.__cost * 0.06
        elif self.__type == 3:
            self.__premium = self.__cost * 0.08

    def get_premium(self):
        return self.__premium


# Global scope variables
vehicle_cost = float(input("Enter the vehicle cost: "))
vehicle_type = int(input("Enter the type of the vehicles (1 for 2 wheeler, 2 for 4 wheeler and
3 for other types): "))

# Create object and calculate premium
vehicle_obj = Vehicle(vehicle_cost, vehicle_type)
vehicle_obj.calculate_premium()

# Display result
print("The premium amount is:", vehicle_obj.get_premium())
```

21) Classic Curve Members

Memberdit.py

Classic Curve Members

```
class Member:
    def __init__(self, customerId, customerName, emailId):
        self.customerId = customerId
        self.customerName = customerName
        self.emailId = emailId


class GoldMember(Member):
    def __init__(self, customerId, customerName, emailId):
        super().__init__(customerId, customerName, emailId)
        self.discountAmount = 0.0

    def calculateDiscount(self, purchaseAmount):
        self.discountAmount = purchaseAmount - (purchaseAmount * 0.10)


class DiamondMember(Member):
    def __init__(self, customerId, customerName, emailId):
        super().__init__(customerId, customerName, emailId)
        self.discountAmount = 0.0

    def calculateBalance(self, purchaseAmount):
        self.discountAmount = purchaseAmount - (purchaseAmount * 0.20)


# Menu
print("1.GoldMembership")
print("2.DiamondMembership")
choice = int(input("Enter the choice\n"))

if choice == 1:
    print("Enter the Details:")
    customerId = input("Customer Id\n")
    customerName = input("Customer Name\n")
    emailId = input("EmailId\n")
    purchaseAmount = float(input("Enter the Purchase Amount\n"))

    g_account_obj = GoldMember(customerId, customerName, emailId)
    g_account_obj.calculateDiscount(purchaseAmount)
```

```
    print("\nMember Details")
    print(g_account_obj.customerId, g_account_obj.customerName, g_account_obj.emailId,
g_account_obj.discountAmount)

elif choice == 2:
    print("Enter the Details:")
    customerId = input("Customer Id\n")
    customerName = input("Customer Name\n")
    emailId = input("EmailId\n")
    purchaseAmount = float(input("Enter the Purchase Amount\n"))

    d_account_obj = DiamondMember(customerId, customerName, emailId)
    d_account_obj.calculateBalance(purchaseAmount)

    print("\nMember Details")
    print(d_account_obj.customerId, d_account_obj.customerName, d_account_obj.emailId,
d_account_obj.discountAmount)

else:
    print("Invalid choice")
```

22) Area calculation - Method Overriding

Shape.py

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def calculate_area(self):
        pass
```

Circle.py

```
import math
from shape import Shape

class Circle(Shape):
    def __init__(self, radius):
        self.__radius = float(radius)

    def calculate_area(self):
        return math.pi * self.__radius ** 2
```

```
    def get_radius(self):
        return self.__radius

    def set_radius(self, radius):
        self.__radius = float(radius)
```

Rectangle.py

```
from shape import Shape

class Rectangle(Shape):
    def __init__(self, length, breadth):
        self.__length = float(length)
        self.__breadth = float(breadth)

    def calculate_area(self):
        return self.__length * self.__breadth

    def get_length(self):
        return self.__length

    def set_length(self, length):
        self.__length = float(length)

    def get_breadth(self):
        return self.__breadth

    def set_breadth(self, breadth):
        self.__breadth = float(breadth)
```

triangle.py

```
from shape import Shape

class Triangle(Shape):
    def __init__(self, base, height):
        self.__base = float(base)
        self.__height = float(height)

    def calculate_area(self):
        return 0.5 * self.__base * self.__height

    def get_base(self):
        return self.__base
```

```
    def set_base(self, base):
        self.__base = float(base)

    def get_height(self):
        return self.__height

    def set_height(self, height):
        self.__height = float(height)
```

main.py

```
# Import necessary modules here
from circle import Circle
from rectangle import Rectangle
from triangle import Triangle

def main():
    shape_type = input("Enter the type of shape: ").strip().lower()

    if shape_type == "circle":
        radius = float(input("Enter the radius: "))
        shape = Circle(radius)
        print(f"The area of Circle is: {shape.calculate_area():.2f}")

    elif shape_type == "rectangle":
        length = float(input("Enter the length: "))
        breadth = float(input("Enter the breadth: "))
        shape = Rectangle(length, breadth)
        print(f"The area of the Rectangle is: {shape.calculate_area():.2f}")

    elif shape_type == "triangle":
        base = float(input("Enter the base: "))
        height = float(input("Enter the height: "))
        shape = Triangle(base, height)
        print(f"The area of Triangle is: {shape.calculate_area():.2f}")

    else:
        print("Invalid shape type entered.")

if __name__ == "__main__":
    main()
```

23) Tournament - Method overloading

Tournament.py

```
# You need to install multipledispatch via:
#    pip install multipledispatch

from multipledispatch import dispatch

# Overload for Cricket: 3 ints → sixers, fours, singles
@dispatch(int, int, int)
def calculateScore(sixers, fours, singles):
    return (sixers * 6) + (fours * 4) + singles

# Overload for Carrom: 2 ints + str → white pockets, black pockets, red pocket flag
@dispatch(int, int, str)
def calculateScore(white, black, red_flag):
    score = (white * 2) + (black * 1)
    if red_flag.lower() == "yes":
        score += 5
    return score

# Overload for Basketball: 2 ints → goals, fouls
@dispatch(int, int)
def calculateScore(goals, fouls):
    return goals - fouls



def main():
    # ------ CRICKET ------
    print("CRICKET:")
    sixers = int(input("Enter the number of sixers: "))
    fours   = int(input("Enter the number of fours: "))
    singles = int(input("Enter the number of singles: "))
    print(f"Calculated Score: {calculateScore(sixers, fours, singles)}")

    # Blank line before next section
    print()

    # ------ CARROM ------
    print("CARROM:")
    white = int(input("Enter the number of white coins pocket: "))
    black = int(input("Enter the number of black coins pocket : "))
```

```
    red   = input("Red Pocket(yes/no): ")
    print(f"Calculated score: {calculateScore(white, black, red)}")

    # Blank line before next section
    print()

    # ------ BASKETBALL ------
    print("BASKETBALL:")
    goals = int(input("Enter the number of goals: "))
    fouls = int(input("Enter the number of fouls: "))
    print(f"Calculated Score: {calculateScore(goals, fouls)}")


if __name__ == "__main__":
    main()
```

24) Complex Addition

Complex.py

```
class Addition:
    def __init__(self):
        self.__real = 0
        self.__img = 0

    def get_real(self):
        return self.__real

    def set_real(self, real):
        self.__real = real

    def get_img(self):
        return self.__img

    def set_img(self, img):
        self.__img = img

    def addRealPart(self, obj1, obj2):
        # Add the real parts of the two objects
        real_sum = obj1.get_real() + obj2.get_real()
        return real_sum
```

```
    def addImaginaryPart(self, obj1, obj2):
        # Add the imaginary parts of the two objects
        img_sum = obj1.get_img() + obj2.get_img()
        return img_sum

    def __add__(self, oth_obj):
        # Create a new object to store the result
        result_obj = Addition()

        # Calculate the sum of real parts and set it in the new object
        new_real = self.get_real() + oth_obj.get_real()
        result_obj.set_real(new_real)

        # Calculate the sum of imaginary parts and set it in the new object
        new_img = self.get_img() + oth_obj.get_img()
        result_obj.set_img(new_img)

        return result_obj

    def __str__(self):
        # Return the complex number in the format "a + bi"
        return f"{self.get_real()} + {self.get_img()}i"

def main():
    # Create two objects of the Addition class
    c1 = Addition()
    c2 = Addition()

    # Get input for the first complex number
    print("Enter the real and imaginary parts of the first complex number:")
    r1 = int(input())
    i1 = int(input())
    c1.set_real(r1)
    c1.set_img(i1)

    # Get input for the second complex number
    print("Enter the real and imaginary parts of the second complex number:")
    r2 = int(input())
    i2 = int(input())
    c2.set_real(r2)
    c2.set_img(i2)

    # Call the specific methods for adding real and imaginary parts
```

```
    real_part_sum = c1.addRealPart(c1, c2)
    imaginary_part_sum = c1.addImaginaryPart(c1, c2)

    print(f"Real part addition: {real_part_sum}")
    print(f"Imaginary part addition: {imaginary_part_sum}")

    # Use the overloaded '+' operator to add the two complex number objects
    result_complex = c1 + c2

    # The __str__ method is called automatically when printing the object
    print(f"Complex Number: {result_complex}")

if __name__ == "__main__":
    main()
```

25) Basket Ball Player Analysis

Main.py

```
import numpy as np

# Create an empty list to store the points
points_list = []

# Prompt the user to enter the points for all 4 quarters
print("Enter John's points for each quarter:")

# Use nested loops to get 4 points for each of the 4 quarters
for i in range(1, 5):  # Loop for quarters 1 to 4
    for j in range(1, 5):  # Loop for points 1 to 4
        # Get user input for each point and convert it to an integer
        point = int(input(f"Quarter {i}, Point {j}: "))
        points_list.append(point)

# Create a 4x4 NumPy array from the list of points
john_statistics = np.array(points_list).reshape(4, 4)

# --- Calculations ---
```

```
# 1. Calculate the total points for the first two quarters (rows 0 and 1)
total_first_two_quarters = john_statistics[0:2].sum()

# 2. Calculate the average points for the last quarter (row 3)
avg_last_quarter = john_statistics[3].mean()


# --- Display Output ---

# Print the final statistics in the required format
print("\nJohn's Statistics:")
print(john_statistics)

print(f"\nTotal points scored by John in the first 2 quarters : {total_first_two_quarters:.2f}")
print(f"Average points scored by John in the last quarter: {avg_last_quarter:.2f}")
```

26) Percentage of marks - Lamda Functions

Percentage.py

```
# Get 3 marks from the user and store them in the specified variables
subject1 = int(input("Enter marks for subject1: "))
subject2 = int(input("Enter marks for subject2: "))
subject3 = int(input("Enter marks for subject3: "))

# Create a lambda function assigned to the variable 'cal'
# This function takes three arguments and returns the percentage
# Formula: (s1 + s2 + s3) / 3 which simplifies to (s1 + s2 + s3) / 3
cal = lambda s1, s2, s3: (s1 + s2 + s3) / 3

# Call the lambda function with the subject marks to get the percentage
percentage_result = cal(subject1, subject2, subject3)

# Display the calculated percentage on the console
print(f"Percentage is {percentage_result}")
```

27) Employee Appraisal

Main.py

```
# A list to hold employee data in memory
employee_data = []

# --- 1. Read the file and calculate new salaries ---
try:
    with open('employee.txt', 'r') as file:
        for line in file:
            # Split each line into parts: id, name, salary
            parts = line.strip().split(',')
            emp_id = parts[0]
            emp_name = parts[1]
            current_salary = float(parts[2])

            # Increment salary by 10%
            new_salary = current_salary * 1.1

            # Store the updated details in our list
            employee_data.append([emp_id, emp_name, new_salary])

    # --- 2. Overwrite the file with updated details ---
    with open('employee.txt', 'w') as file:
        for emp in employee_data:
            # Write each employee's updated data, formatting the salary to 2 decimal places
            file.write(f"{emp[0]},{emp[1]},{emp[2]:.2f}\n")

    # --- 3. Display the updated details ---
    print("The updated details are:")
    for emp in employee_data:
        print(f"{emp[0]},{emp[1]},{emp[2]:.2f}")

    print() # Adds a blank line for formatting

    # --- 4. Search for an employee ---
    search_id = input("Enter the employee id: ")
    found = False

    # Read the updated file again to perform the search
    with open('employee.txt', 'r') as file:
        for line in file:
            parts = line.strip().split(',')
            emp_id_from_file, emp_name, emp_salary = parts

            # Check if the current line's ID matches the search ID
```

```
            if emp_id_from_file == search_id:
                print(f"The employee {emp_name} with id {emp_id_from_file} has salary
Rs.{emp_salary}")
                found = True
                break # Stop searching once the employee is found

    # If the loop finishes and the employee was not found
    if not found:
        print("No records found")

except FileNotFoundError:
    print("Error: 'employee.txt' not found. Please create the file with employee data.")
```

28) Product Information

Main.py

```
# Step 1: Read content from 'toys.txt' and copy to 'new_toys.txt'
try:
    with open("toys.txt", "r") as old_file:
        old_toys = old_file.readlines()
except FileNotFoundError:
    print("Error: 'toys.txt' not found.")
    old_toys = []

with open("new_toys.txt", "w") as new_file:
    for toy in old_toys:
        new_file.write(toy.strip() + "\n")

# Step 2: Take input for new toys
num_new_toys = int(input("Enter the no. of new toys: "))
print("Enter the toy's details:")
new_toys = []
for _ in range(num_new_toys):
    toy_detail = input()
    new_toys.append(toy_detail)

# Step 3: Append new toys to 'new_toys.txt'
with open("new_toys.txt", "a") as new_file:
    for toy in new_toys:
        new_file.write(toy.strip() + "\n")
```

```
# Step 4: Display all toy details and count
print("\nToy Details:")
with open("new_toys.txt", "r") as new_file:
    all_toys = [line.strip() for line in new_file.readlines()]
    for toy in all_toys:
        print(toy)

print("\nNo. of toys:", len(all_toys))
```

29) Filter Customers - JSON File

Loan.py

```
import json

def filter_customers(loan_type):
    try:
        with open('loan.json', 'r') as file:
            data = json.load(file)

        customers = data.get("customers", [])
        filtered = []

        for customer in customers:
            for loan in customer.get("loans", []):
                if loan.get("loan_type", "").lower() == loan_type.lower():
                    filtered.append({
                        "Account_Number": customer.get("Account_Number"),
                        "Customer_Name": customer.get("customer_name"),
                        "Loan_Amount": loan.get("loan_amount")
                    })

        if filtered:
            print("Loan Details")
            print("Account_Number     Customer_Name   Loan_Amount")
            for entry in filtered:
                print(f"{entry['Account_Number']}       {entry['Customer_Name']}
{entry['Loan_Amount']}")
        else:
            print("No customers available")

    except FileNotFoundError:
        print("loan.json file not found.")
    except json.JSONDecodeError:
```

```
        print("Error decoding JSON file.")

# Get loan_type input from user
loan_type = input("Enter the loan type\n")
filter_customers(loan_type)
```

30) Filter the Countries Sachin has Played Against - CSV file

FilterCountries.py

```
import csv

# Set to store unique countries Sachin played against
versus_countries = set()

# Read the CSV file using DictReader
with open("OneDayInternational.csv", mode="r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        # Check if the player is Sachin Tendulkar
        if row["Player"].strip() == "Sachin R Tendulkar":
            country = row["Versus"].strip()
            versus_countries.add(country)

# Sort countries alphabetically
sorted_countries = sorted(list(versus_countries))

# Print the result
print("Countries Sachin played against (1990-1998):")
for country in sorted_countries:
    print(country)
```

31) Filter Customers - JSON File

Loan.py

```
import json

# Step 1: Get Loan type from user
loan_type = input("Enter the loan type\n").strip().lower()

# Step 2: Define the function to filter customers
def filter_customers(loan_type):
```

```
        try:
                with open("loan.json", "r") as file:
                        data = json.load(file)
        except FileNotFoundError:
                print("Error: 'loan.json' file not found.")
                return

        customers_found = []

        for customer in data.get("customers", []):
                for loan in customer.get("loans", []):
                        if loan.get("loan_type", "").lower() == loan_type:

                                customers_found.append({
                                        "Account Number": customer["Account_Number"],

                                        "Customer Name": customer["customer_name"],
                                        "Loan_Amount": loan ["loan_amount"]
                                })
        if customers found:
                print("\nLoan Details")
                print("Account_Number Customer Name Loan_Amount")
                for c in customers_found:
                        print(f"{c['Account_Number']} (c['Customer Name'])
(c['Loan_Amount"]}

        else:
                print("No customers available")

# Step 3: Call the function

filter_customers (loan_type)
```

32) Data Encryption

Encryption.py

```
import re

def encrypt_message(message):
   def shift_char(match):
      ch = match.group()
      if ch.isalpha():
```

```
            if ch.islower():
                return chr((ord(ch) - ord('a') + 1) % 26 + ord('a'))
            else:
                return chr((ord(ch) - ord('A') + 1) % 26 + ord('A'))
        return ch

    # Replace each alphabetic character using regex
    encrypted = re.sub(r'[A-Za-z]', shift_char, message)

    # Replace digits with their incremented value
    encrypted = re.sub(r'\d', lambda x: str((int(x.group()) + 1) % 10), encrypted)

    return encrypted

# ----------- Main Program -----------

message = input("Actual message: ")
encrypted = encrypt_message(message)
print("Encrypted message:", encrypted)
```

33) Project Allocation – Iterator

Allocation.py

```
class ProjectAllocation:
    def __init__(self, score_list):
        #write your code here
        self.score_list = score_list
        self.index = 0

    #Define necessary member methods needed for an iterator
    def __iter__(self):
        #write your code here
        self.index = 0
        return self

    def __next__(self):
        #write your code here
        if self.index >= len(self.score_list):
            raise StopIteration
        emp_data = self.score_list[self.index]
        emp_id = emp_data[0]
        scores = emp_data[1:]
```

```
        avg_score = sum(scores) / len(scores)
        self.index += 1
        return {emp_id: round(avg_score, 1)}
```

main.py

```
#import necessary modules here
from allocation import ProjectAllocation

def main():
    # Write your code here for getting user inputs, creating object for the custom iterator 'ProjectAllocation'
    # and displaying the result using iterator.

    num_employees = int(input("Enter the no. of employees:"))
    score_list = []

    for _ in range(num_employees):
        emp_id = input("Enter the employee id:")
        scores_input = input("Enter the scores of 5 assessments (comma separated) for the employee:")
        scores = list(map(int, scores_input.strip().split(',')))
        score_list.append([emp_id] + scores)

    print("Employees' average scores:")
    allocation = ProjectAllocation(score_list)
    for result in allocation:
        print(result)

    return None

if __name__ == "__main__":
    main()
```

34. Summer Camp – Decorator

Main.py

```
import re

def validate(func):
    def inner(candidates):
        #Write your code here
```

```
        valid_list = []
        pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
        for candidate in candidates:
            if re.match(pattern, candidate[2]):
                valid_list.append(candidate)
        return func(valid_list)
    return inner

@validate
def segregate(candidates):
    #Write your code here
    under_10 = 0
    under_15 = 0
    under_20 = 0
    for candidate in candidates:
        age = int(candidate[1])
        if 5 <= age < 10:
            under_10 += 1
        elif 10 <= age < 15:
            under_15 += 1
        elif 15 <= age < 20:
            under_20 += 1
    return under_10, under_15, under_20

def main():
    candidates = []
    #Write your code here
    while True:
        detail = input("Enter the candidate details as (name:age: email): ")
        parts = detail.strip().split(":")
        candidates.append(parts)
        cont = input("Do you want to enter another candidate's details(y/n): ")
        if cont.lower() != 'y':
            break

    u10, u15, u20 = segregate(candidates)
    print("No. of participants under 10:", u10)
    print("No. of participants under 15:", u15)
    print("No. of participants under 20:", u20)

    return

if __name__=='__main__':
```

main()